

```

1  (*
2      CS 51 Lab 19
3      Brian's Brain Cellular Automaton
4      https://en.wikipedia.org/wiki/Brian's_Brain
5  *)
6
7  module G = Graphics ;;
8
9  (* Automaton parameters *)
10 let cGRID_SIZE = 100 ;; (* width and height of grid in cells *)
11 let cSPARSITY = 5 ;; (* inverse of proportion of cells initially live *)
12
13 (* Rendering parameters *)
14 let cCOLOR_LIVE = G.rgb 200 200 200 ;; (* color to depict live cells *)
15 let cCOLOR_DYING = G.rgb 130 0 0 ;; (* color to depict dying cells *)
16 let cCOLOR_DEAD = G.rgb 0 0 0 ;; (* background color *)
17 let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
18 let cSIDE = 8 ;; (* width and height of cells in pixels *)
19 let cRENDER_FREQUENCY = 1 (* how frequently grid is rendered (in ticks) *) ;;
20 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
21
22 (*-----
23 Brian's Brain Cellular Automaton
24 *)
25
26 (*.....
27 States, updates, and utility functions
28 *)
29
30 (* We take the opportunity to abstract the cell states as an
31 enumerated type *)
32 type bbrain_state =
33 | Live
34 | Dying
35 | Dead ;;
36
37 (* offset index off -- Returns the `index` offset by `off` allowing
38 for wraparound. *)
39 let rec offset (index : int) (off : int) : int =
40   if index + off < 0 then
41     cGRID_SIZE - (offset ~-index ~-off)
42   else
43     (index + off) mod cGRID_SIZE ;;
44
45 (* bbrain_update grid i j -- Returns the updated value for cell at `i,
46 j` in the grid based on rules of Brian's Brain
47 <https://en.wikipedia.org/wiki/Brian%27s_Brain>. *)
48 let bbrain_update (grid : bbrain_state array array)
49   (i : int) (j : int)
50   : bbrain_state =
51   let neighbors = ref 0 in
52   for i' = ~-1 to ~+1 do

```

```

53   for j' = ~-1 to ~+1 do
54     let i_ind = offset i i' in
55     let j_ind = offset j j' in
56     neighbors := !neighbors
57               + match grid.(i_ind).(j_ind) with
58                 | Live -> 1
59                 | Dying -> 0
60                 | Dead -> 0
61   done
62 done;
63 (* determine new state based on neighbor count *)
64 let new_state =
65   match grid.(i).(j) with
66   | Live -> Dying
67   | Dying -> Dead
68   | Dead -> if !neighbors = 2 then Live else Dead in
69   new_state ;;
70
71 (* bbrain_initialize grid -- Fills `grid` with a random mix of Live and
72    Dead cells (Dying cells are never seeded; they arise naturally on the
73    next tick), with roughly 1-in-cSPARSITY cells initially live. *)
74 let bbrain_initialize (grid : bbrain_state array array) : unit =
75   for i = 0 to cGRID_SIZE - 1 do
76     for j = 0 to cGRID_SIZE - 1 do
77       grid.(i).(j) <- if Random.int cSPARSITY = 0 then Live else Dead
78     done
79   done
80
81 (* .....
82    Making Brian's Brain
83    *)
84
85 (* Automaton specification *)
86
87 module BBrainSpec : (Cellular.AUT_SPEC
88                     with type state = bbrain_state) =
89   struct
90     type state = bbrain_state
91     let initial : state = Dead
92     let grid_size : int = cGRID_SIZE
93     let initialize = bbrain_initialize
94     let update = bbrain_update
95     let name : string = "Brian's Brain"
96     let cell_color (cell : state) : G.color =
97       match cell with
98       | Live -> cCOLOR_LIVE
99       | Dying -> cCOLOR_DYING
100      | Dead -> cCOLOR_DEAD
101     let side_size : int = cSIDE
102     let legend_color : G.color = cCOLOR_LEGEND
103     let font : string option = cFONT
104     let render_frequency : int = cRENDER_FREQUENCY
105   end ;;
106

```

```

107 module Aut = Cellular.Automaton (BBrainSpec) ;;
108
109 (* .....
110    Running the automaton and displaying the results
111    *)
112
113 (* main seed -- Runs the automaton using the provided random `seed`
114    (for replicability), displaying updates to the graphics window. *)
115 let main seed =
116
117     Random.init seed;      (* initialize randomness *)
118     Aut.graphics_init ();  (* ... and graphics window *)
119
120     Aut.run_grid (Aut.initialized_grid ()) ;;
121
122 (* Run the automaton with user-supplied seed *)
123 let _ =
124     if Array.length Sys.argv <= 1 then
125         Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
126             Sys.argv.(0)
127     else
128         main (int_of_string Sys.argv.(1)) ;;

```